

lab2.pdf

任务介绍

本次实验在实验1构建的AST基础上，构建符号表，并进行类型检查。

功能实现

符号表

其中，符号表采用函数式实现

```
1 std::vector<std::unordered_map<std::string, SymbolPtr>> scopes;
```

每次进入新的作用域时，调用 `enter_scope` 方法，在 `scopes` 向量中添加一个新的空符号表：

```
1 void SymbolTable::enter_scope() {  
2     scopes.push_back({});  
3 }
```

每次离开当前作用域时，调用 `exit_scope` 方法，从 `scopes` 向量中移除当前符号表：

```
1 void SymbolTable::exit_scope() {  
2     if (!scopes.empty()) {  
3         scopes.pop_back();  
4     }  
5 }
```

在当前作用域中添加符号时，调用 `add_symbol` 方法，将符号添加到 `scopes` 向量的最后一个符号表中，同时生成唯一名称（全局作用域保持原名，局部作用域添加后缀）

从当前作用域向上查找符号时，调用 `find_symbol` 方法，遍历 `scopes` 向量，从最后一个符号表开始查找（即从内往外找，实现值按作用域的覆盖）。

类型检查

类型检查的核心思想是递归地遍历AST，对每个节点进行类型推导和检查，并验证类型约束。主要步骤包括：

1. **符号检查**：确保变量/函数在使用前已定义，处理作用域关系
2. **类型兼容性**：确保表达式、赋值、参数传递等操作类型兼容
3. **特殊规则检查**：如数组访问、初始化列表、控制流语句等
4. **类型推导**：根据节点结构和子节点类型确定当前节点的类型

具体内部实现比较多而繁杂，这里只展示一下我的**迭代过程**：

1. 添加符号表，完全不检查只添加，打印查看
2. 完成函数定义唯一性检查添加+函数参数返回值类型创建
3. 初步完成checkVarDef，对同一作用域下变量重复定义检查通过
4. 修复进入函数时的作用域双重创建问题，完成形参与局部变量的重名检查
5. 完成checkLval,完成对未定义的变量的使用检查
6. 完成checkFunCall,实现未定义函数使用检查
7. 实现checkAssignStmt，为方便拓展，实现Lval是否为数组与Lval type是否等于expr type双重检查，方便之后支持其他的数据类型
8. 初步实现checkReturnStmt，但发现缺乏直接的跟踪当前正在检查的函数的返回类型的方式（查符号表并不可靠，因为可能有多个函数，没法确定是哪个）
9. 添加TypePtr current_function_return_type跟踪当前函数返回类型，对应修改构造函数，checkFuncDef，checkReturnStmt，实现返回值检查
10. 实现checkUnaryExp，checkBinaryExp，计算一元二元表达式类型，以供检查
11. 完善checkVarDef，添加对数组初始化的检查
12. 尝试通过完善checkBinaryExp通过if-else-return测试，但失败了
13. 部分完善数组初始化检查，禁止使用标量初始化数组
14. 添加并实现checkIfStmt，通过if-else-return测试
15. 为了完整性，添加并实现checkWhileStmt，虽然好像没有相应的没通过的测试
16. 设计了While的错误测试
17. 添加了checkArrayInitList并修改checkVarDef来实现多维数组的初始化过程检查，对齐和贪心填充规则
18. 拉取最新测试用例，发现新增了两个不通过的，其中一个是语法分析不通过
19. 修改.y文件支持非数组类型采用列表初始化
20. 修改checkVarDef，允许int类型用只含一个元素的初始化列表初始化
21. checkVarDef相关代码继续修改+重构，支持复杂的数组初始化检查

实现中的亮点

涉及到新节点添加和全局修改的，主要就是标黄的这几项，其他的都是在模版代码下实现的。

添加TypePtr current_function_return_type是为了追踪当前函数返回值到底应该是啥类型，因为函数返回值这个东西查符号表没法确定，因为函数定义可以有多个，虽然可能硬是要去设计可能...也可以？但感觉干脆加个成员变量好做很多。

If 和While不必多说，都需要加一个专门的check，同时发现测试用例里缺少while块内的非法用例，就补充提交了一个；

非数组类型采用列表初始化这个东西当时在做lab1的时候就没注意到，也没测出来，所以从语法分析开始一路改；

checkVarDef这里应该算是整个实现最复杂的地方：数组初始化检查。

数组初始化检查

数组初始化检查的核心函数是 `checkArrayInitList`，它采用递归设计，能够处理嵌套的初始化列表。其基本思路是：

- 追踪已使用的元素数量
- 处理嵌套初始化列表与维度的对应关系
- 验证类型兼容性
- 检查边界情况

处理流程：

1. 首先计算数组总元素数和每个维度的"步长":
 - 最内层维度步长为1
 - 外层维度步长为内层维度的乘积
2. 再用"维度对齐"处理嵌套的初始化列表：
 - a. 从外层维度开始，查找可以对齐的维度
 - b. 如果找到，则递归处理子数组
 - c. 处理完子数组后，将元素计数对齐到维度边界
3. 对于非嵌套元素(如简单的整数)，主要检查：
 - 表达式类型是否合法
 - 不允许将数组类型赋值给标量元素(如案例 `array_init_error4.sy` 中的错误)
4. 一些边界检查，略

AI工具与代码借鉴使用声明

本实验本人没有借鉴任何除统一提供的模版之外的代码。

本报告没有使用AI工具编写。

本实验的代码部分由本人进行构思和调试以及设计性编写，而部分结构重复的节点定义由AI（github copilot）生成。

建议和想法

我觉得本次实验的指导书和模版仍然非常的全面和友好，有效地平滑了实验的难度曲线，感谢助教为此做出的贡献！